

Лабораторная работа № 14: Исследование методов работы с файловой системой средствами Node.js.

Цель работы: Сформировать умение анализировать методы работы с файловой системой средствами [Node.js](#).

Общие требования ко всем заданиям

1. Использовать только асинхронные методы с `async/await` (модуль `fs.promises`)
 2. Использовать модуль `path` для всех операций с путями
 3. Реализовать обработку всех возможных ошибок
 4. Для файлов > 1 МБ использовать потоки
 5. Все пути должны быть относительными к текущей директории
 6. Код должен быть хорошо прокомментирован
-

1. Подготовка рабочего окружения

1.1 Обновление основной ветки

```
bash
```

```
git checkout main
```

```
git pull origin main
```

1.2 Создание новой ветки для работы

```
bash
```

```
git checkout -b feature/createfiles
```

Задание 1. Создание и чтение файлов

Условие:

Создайте программу, которая:

1. Создает файл `student_N.txt`, где N - номер вашего варианта
2. Записывает в него следующую информацию:
 - Фамилия и имя студента
 - Номер группы
 - Номер варианта
 - Текущая дата и время
 - Список из 5 ваших любимых книг/фильмов (каждый с новой строки)
3. Добавляет в конец файла строку "Количество записей: X", где X - количество строк в файле
4. Читает файл и выводит его содержимое в консоль в отформатированном виде

Пример выполнения (Вариант 5):

text

Создан файл: `student_5.txt`

Содержимое файла:

Студент: Иванов Иван

Группа: ИС-202

Вариант: 5

Дата: 2026-09-09 14:30:25

Любимые книги:

1. "Война и мир" - Л. Толстой
2. "Преступление и наказание" - Ф. Достоевский
3. "Мастер и Маргарита" - М. Булгаков
4. "1984" - Дж. Оруэлл
5. "Гарри Поттер" - Дж. Роулинг

Количество записей: 10

Задание 2. Работа с каталогами

Условие:

Создайте программу, которая:

1. Создает следующую структуру каталогов в папке `project_N` (где N - номер варианта):

text

```
project_N/  
├── src/  
│   ├── modules/  
│   ├── components/  
│   └── utils/  
├── data/  
│   ├── input/  
│   └── output/  
└── temp/
```

2. В каждой из созданных папок создает файл `info.txt` с описанием назначения папки
3. Выводит в консоль полное дерево созданной структуры
4. Перемещает папку `temp` внутрь папки `data`
5. Переименовывает папку `data/output` в `data/results`
6. Удаляет папку `temp` со всем содержимым
7. Выводит обновленное дерево структуры

Дополнительное условие (зависит от варианта):

- Если вариант четный: в каждой папке создать дополнительный файл `README.md` с текущей датой
- Если вариант нечетный: в папке `src/components` создать 3 вложенные папки с номерами от 1 до 3

Задание 3. Поиск и фильтрация файлов

Условие:

Создайте программу, которая:

1. Принимает путь к директории через аргументы командной строки (если не указан - используется текущая)
2. Сканирует все файлы и папки в указанной директории (рекурсивно)
3. Выводит статистику:
 - Общее количество файлов
 - Общее количество папок
 - Общий размер всех файлов (в байтах, КБ, МБ)
 - Список всех файлов с расширениями (сгруппированных по типу)
 - Топ-5 самых больших файлов
 - Топ-5 самых маленьких файлов
4. Создает файл отчета `report_N.json` (где N - номер варианта) с этой статистикой

Дополнительное условие (зависит от варианта):

- Варианты 1-5: Игнорировать файлы размером более 10 МБ
- Варианты 6-10: Игнорировать файлы в папках `node_modules` и `.git`
- Варианты 11-15: Анализировать только файлы с расширениями `.js`, `.json`, `.txt`, `.md`
- Варианты 16-20: Добавить поиск файлов, содержащих в названии номер варианта

Пример вывода:

text



Анализ директории: `./my_project`



Общее количество папок: 15



Общее количество файлов: 42



Общий размер: 2.47 МБ (2,590,720 байт)



Расширения файлов:

`.js`: 12 файлов (1.2 МБ)

`.json`: 8 файлов (856 КБ)

`.css`: 5 файлов (245 КБ)

`.html`: 4 файла (128 КБ)

`.txt`: 3 файла (41 КБ)

.md: 10 файлов (12 КБ)

🏆 Топ-5 самых больших файлов:

1. bundle.js (1.8 МБ) - ./dist/bundle.js
2. data.json (456 КБ) - ./data/data.json
3. styles.css (234 КБ) - ./src/styles.css
4. image.png (123 КБ) - ./assets/image.png
5. config.json (89 КБ) - ./config.json

📄 Отчет сохранен: report_5.json

Задание 4. Потокковая обработка данных (Сложное)

Условие:

Создайте программу для обработки большого текстового файла с использованием потоков:

1. Генерация файла (если файл не существует):
 - Создать файл `data_N.txt` (где N - номер варианта) размером минимум 100 000 строк
 - Каждая строка содержит: номер строки, случайное число от 1 до 1000, текст "Вариант N"
 - Пример: 1, 847, Вариант 5
2. Обработка файла через потоки:
 - Прочитать файл через поток
 - Для каждой строки вычислить:
 - Сумму всех чисел
 - Среднее арифметическое
 - Максимальное и минимальное число
 - Количество строк
 - Создать новый файл `processed_N.txt` с результатами
 - Выводить прогресс обработки в процентах (каждые 10%)
3. Работа с большим файлом (если файл > 1 МБ):
 - Использовать потоковое чтение с буфером 64 КБ
 - Обработку выполнять без загрузки всего файла в память

Дополнительное условие (зависит от варианта):

- Варианты 1-5: Дополнительно подсчитать количество четных и нечетных чисел
- Варианты 6-10: Дополнительно найти 10 самых часто встречающихся чисел
- Варианты 11-15: Дополнительно записать в новый файл только строки с числами больше 500
- Варианты 16-20: Дополнительно вычислить медиану всех чисел

Пример вывода:

text

 Обработка файла: data_5.txt

Размер файла: 2.3 МБ

 Прогресс: 10% (10,000 строк обработано)

 Прогресс: 20% (20,000 строк обработано)

...

 Прогресс: 100% (100,000 строк обработано)

 Обработка завершена!

 Результаты:

- Всего строк: 100,000
- Сумма чисел: 50,234,567
- Среднее значение: 502.35
- Максимальное число: 999
- Минимальное число: 1
- Четных чисел: 50,123
- Нечетных чисел: 49,877

 Результаты сохранены в: processed_5.txt

 Время выполнения: 1.23 сек

Задание 5. Копирование и синхронизация (Сложное)

Условие:

Создайте программу для резервного копирования с использованием потоков:

1. Создание тестовой структуры:
 - Создать папку `source_N` (N - номер варианта)
 - Создать в ней 20 файлов с разными расширениями и размерами
 - Создать 3 подпапки с файлами внутри
 - Создать файл `manifest.json` с описанием всех файлов
2. Копирование с фильтрацией:
 - Скопировать все файлы из папки `source_N` в папку `backup_N`
 - При копировании:
 - Файлы с расширениями `.txt`, `.js`, `.json` копировать через потоки
 - Файлы с расширениями `.jpg`, `.png`, `.gif` копировать обычным способом
 - Файлы размером > 1 МБ разделять на чанки по 512 КБ
 - Сохранять структуру папок
 - Выводить прогресс копирования
3. Синхронизация:
 - Реализовать функцию сравнения `source_N` и `backup_N`
 - Найти добавленные, удаленные и измененные файлы
 - Создать отчет `sync_report_N.txt` с результатами сравнения

Дополнительное условие (зависит от варианта):

- Варианты 1-5: Добавить сжатие всех текстовых файлов при копировании (удаление лишних пробелов)
- Варианты 6-10: Добавить проверку контрольных сумм (MD5) для файлов > 500 КБ
- Варианты 11-15: Реализовать инкрементальное копирование (только измененные файлы)
- Варианты 16-20: Добавить архивацию в ZIP с использованием стороннего модуля

Пример вывода:

text

📁 Исходная директория: `source_5`

📁 Директория назначения: `backup_5`

📋 Обнаружено файлов: 45

└─ 📄 `.txt`: 12 файлов (1.2 МБ) - потоковое копирование

└─ 📄 `.js`: 8 файлов (856 КБ) - потоковое копирование

- └─  .json: 5 файлов (245 КБ) - потоковое копирование
- └─  .jpg: 10 файлов (5.6 МБ) - обычное копирование
- └─  другие: 10 файлов (2.3 МБ) - обычное копирование

 Прогресс копирования: 45/45 файлов

 Копирование завершено!

 Статистика:

- Скопировано файлов: 45
- Потоковое копирование: 25 файлов
- Обычное копирование: 20 файлов
- Общий размер: 10.2 МБ
- Время выполнения: 2.45 сек

 Сравнение директорий:

- Совпадают: 42 файла
- Изменены: 2 файла (size, modified)
- Добавлены: 1 файл
- Удалены: 0 файлов

 Отчет сохранен: sync_report_5.txt

Фиксация изменений

Добавить файлы в Git

Создать коммит

Отправить в удалённый репозиторий

Создать Pull Request